

MORE THAN JUST WEIGHT

Skeletal Mesh 시스템 구현 설명 문서

주제	FBX 파싱 · CPU Skinning · Component 설계 · SkeletalMesh Viewer
팀	Team 3
팀원	김연하 · 성원희 · 남윤지 · 김형도
주차	Week 10

01	FBX 구조 및 파싱
02	CPU Skinning
03	Component 설계
04	SkeletalMesh Viewer
05	에셋 파이프라인 (Source → .uasset)

01. FBX 구조 및 파싱

FBX(Filmbox)는 Autodesk가 개발한 3D 모델·애니메이션 표준 파일 포맷으로, 게임 엔진에서 Skeletal Mesh와 애니메이션 데이터를 교환하는 데 가장 널리 사용됩니다. FBX 파일은 계층적 노드 트리로 구성되며, 주요 구성 요소는 아래와 같습니다.

Root Node	씬 전체의 최상위 노드. Mesh Node와 Skeleton이 여기에 연결됩니다.
Mesh Node	버텍스(Vertex), 법선(Normal), UV 좌표 등 메시 기하 데이터를 보유합니다.
Skeleton / Bone	계층 구조로 연결된 뼈대 정보. Neck → Head 처럼 부모-자식 관계를 가집니다.
Deformer: Skin	어떤 Skinning 방식(Linear Blend 등)으로 버텍스를 변형할지 정의합니다.
Deformer: Cluster	개별 Bone이 각 버텍스에 미치는 가중치(Weight), 바인드 포즈 역행렬(Transform), 현재 Bone 로컬 행렬(TransformLink)을 보관합니다.
Connection	Cluster → Bone, Skin → Mesh Node 등 노드 간 참조 관계를 명시합니다.

파싱 흐름 요약

1. FBX SDK로 씬을 로드 → Root Node 탐색

2. Mesh Node에서 Vertex / Normal / UV 추출
3. Skeleton 계층을 재귀 순회하여 Bone 리스트 구성
4. Skin Deformer에서 Cluster 목록 읽기
5. 각 Cluster의 Weight 배열과 Transform / TransformLink 행렬 저장
6. Connection 테이블로 Cluster ↔ Bone 매핑 완성

02. CPU Skinning

Skinning이란 Bone의 현재 포즈(Pose)에 따라 메시 버텍스를 변형하는 과정입니다. CPU Skinning은 이 연산을 CPU에서 수행하며, 다음 공식으로 최종 버텍스 위치를 계산합니다.

$$V_{\text{final}} = \sum (V_{\text{local}} \times M_{\text{offset},i} \times M_{\text{combined},i}) \times W_i$$

<code>V_local</code>	Mesh 공간에서의 버텍스 원본 위치 (바인드 포즈 기준)
<code>M_offset,i</code>	바인드 포즈 시 Bone의 글로벌 Transform 역행렬 → 버텍스를 Bone 로컬 공간으로 변환
<code>M_combined,i</code>	현재 프레임 Bone의 글로벌 Transform 행렬 → Bone 로컬 공간 → 월드 공간 변환
<code>W_i</code>	해당 Bone이 버텍스에 미치는 가중치 (모든 Bone의 가중치 합 = 1)

CPU vs GPU Skinning

구분	CPU Skinning	GPU Skinning
연산 위치	CPU 메인 스레드(또는 워커)	Vertex Shader (GPU)
장점	결과를 CPU에서 즉시 활용 (물리, 충돌)	병렬 처리로 다수 캐릭터 고성능
단점	버텍스 수 많을수록 CPU 부하 증가	CPU 접근 어려움, 구현 복잡도 높음
이번 구현	채택	—

03. Component 설계

엔진 내 Skeletal Mesh 렌더링을 담당하는 두 컴포넌트를 설계하였습니다. 계층 구조로 책임을 분리하여 재사용성과 확장성을 높였습니다.

항목	SkinnedMeshComponent	SkeletalMeshComponent
핵심 역할	Skinning 가능한 Mesh Component의 공통 기반 (Base Class)	Skeleton / Bone 기반 Mesh Component
Transform 공급	외부 시스템 또는 하위 클래스에서 공급	Bone Hierarchy를 기반으로 자체 계산

주요 기능	스키닝 데이터 관리 렌더링 제출 공통 처리	Bone Pose 계산 CPU Skinning 실행 SkeletalMesh 렌더링
-------	-------------------------	---

설계 원칙

- SkinnedMeshComponent는 추상 기반 클래스로, Skinning에 필요한 공통 인터페이스와 버퍼를 제공합니다.
- SkeletalMeshComponent는 이를 상속받아 Bone Hierarchy 순회, Pose 계산, CPU Skinning까지 완전히 처리합니다.
- 외부 애니메이션 시스템(AnimInstance 등)은 SkinnedMeshComponent 레벨에서 Transform을 주입할 수 있어 유연성을 확보합니다.

04. SkeletalMesh Viewer

Asset Editor의 일환으로 구현된 SkeletalMesh Viewer는 임포트된 Skeletal Mesh를 실시간으로 확인하고 편집할 수 있는 독립 창입니다.

기능	설명
뷰포트 (View)	MMB Drag: Orbit / Pan, 휠: Zoom, F: Frame — 카메라 조작으로 메시를 자유롭게 확인
Bone 표시	Show Bones 토글로 뼈대 시각화, 선택한 Bone에 Gizmo(Transform 핸들) 표시
Bone Hierarchy Tree	hips → spine → chest → neck → head 등 전체 Bone 트리를 패널에 출력, 클릭으로 선택
Pose Edit Mode	Pose Edit Mode 활성화 시 개별 Bone을 직접 회전·이동하여 포즈를 수정할 수 있음
메시 통계	Bones: 78, Vertices: 11,640, Indices: 11,640, LOD: 0 등 메시 정보 실시간 표시

05. 에셋 파이프라인: Source → Content (.uasset)

엔진은 원본 파일(Source)을 직접 사용하지 않고, 엔진 전용 포맷인 .uasset으로 변환(Import)하여 사용합니다. 이 파이프라인은 Level Editor와 Asset Editor의 역할 분리를 전제로 합니다.

Level Editor	Asset Editor
게임 레벨(씬)을 구성하는 에디터. Actor 배치, 라이팅, 트리거 설정 등 레벨 전체를 편집합니다.	개별 Asset을 열고 편집하는 에디터. StaticMesh, SkeletalMesh, Material, Texture2D 등을 처리합니다.

FBX Import 흐름

Source (.fbx)	Miku.fbx + 텍스처 PNG 묶음 등 원본 파일
---------------	-------------------------------

Import	엔진 Content Browser에서 Import 버튼 → FBX Importer 실행
파싱 & 변환	FBX SDK로 Mesh / Skeleton / Skin 데이터 추출 후 엔진 내부 구조로 변환
Content (.uasset)	SK_Miku.uasset 생성 + 관련 텍스처 에셋(T_Miku_Hair 등) 자동 생성
Load & 렌더링	SkeletalMeshComponent가 .uasset을 로드 → CPU Skinning → 렌더링

정리 및 향후 과제

- FBX SDK를 활용해 Mesh Node, Skeleton, Skin Deformer, Connection 정보를 완전히 파싱하였습니다.
- CPU Skinning 공식($V_{final} = \sum V_{local} \times M_{offset} \times M_{combined} \times W$)을 구현하여 Bone Pose 변화를 메시에 반영합니다.
- SkinnedMeshComponent / SkeletalMeshComponent 계층으로 책임을 분리하고 확장 가능한 구조를 확보했습니다.
- SkeletalMesh Viewer에서 Bone Hierarchy Tree, Pose Edit Mode, 카메라 조작, 메시 통계를 제공합니다.
- Source → .uasset Import 파이프라인을 Level Editor / Asset Editor 이원 구조로 구현하였습니다.
- 향후 GPU Skinning, AnimBlueprint 연동, LOD 시스템 등을 추가로 구현할 예정입니다.